

Part 3: Software-Defined Vehicle (SDV) Centralization

Migrating from Distributed ECUs to High-Performance Computing (HPC)

1. Paradigm Shift: Legacy vs. SDV

Traditional vehicle electrical architectures are **distributed domain networks**. In this legacy model, each vehicular feature (door lock, wiper, seat motor, inverter, battery management) has its own dedicated micro-controller (ECU) supplied by different Tier-1 vendors with proprietary, static firmware.

A modern luxury vehicle often contains over **100 distributed ECUs**. This leads to massive architectural friction: implementing a feature requiring multiple domains (e.g. automatically locking doors and adjusting suspension when speed increases) requires coordinating software releases from multiple vendors.

Metric	Legacy Distributed Architecture	Software-Defined Vehicle (SDV)
ECU Count	70 - 120+ dedicated microcontrollers	1 - 3 Central Computers + Zonal Gateways
Control Logic	Burned into fixed ECU firmware	Consolidated in central software tasks
Wiring Harness	Heavy point-to-point domain wiring	Short zonal wiring to local edge hubs
Upgradability	Dealership flashing / physical recall	Over-The-Air (OTA) continuous updates
Feature Velocity	Years (hardware development cycle)	Weeks (agile software deployment)

2. Our Dual-Bus Simulation Topology

To faithfully mirror modern vehicle domain isolation, our simulation organizes vehicle functions into two distinct virtual buses:

- **Bus 0 (``vcan0``) - Powertrain & High Voltage:** Connects the Powertrain Control Module (``pcm_node``) and Battery Management System (``bms_node``). High-voltage safety and motor control operate on this isolated bus.
- **Bus 1 (``vcan1``) - Body & Cockpit:** Connects the Body Control Module (``bcm_node``) and Driver Cockpit Interface (``hmi_node``). Cabin lighting, locks, and driver pedal inputs reside here.

Crucially, **Bus 0 and Bus 1 are not physically or virtually bridged**. The **Central Vehicle Computer (``central_compute``)** is the sole dual-homed node connected to both interfaces, enforcing strict safety arbitration, privilege separation, and software routing.

3. Cross-Domain Software Coordination

Consider the **Auto-Speed Door Lock** feature in our SDV simulation:

1. The driver applies accelerator pedal in `hmi\_node` (Bus 1).
2. Central Compute validates battery health from `bms\_node` (Bus 0) and commands torque to `pcm\_node` (Bus 0).
3. As vehicle speed accelerates past 15.0 km/h, Central Compute detects the threshold on Bus 0.
4. Central Compute autonomously dispatches an actuation frame on Bus 1 commanding `bcm\_node` to lock all doors.
5. Neither `pcm\_node` nor `bcm\_node` knew about each other; the entire feature was realized strictly in central software.

Key Takeaway: *This exemplifies the core SDV thesis: Hardware is abstracted into standardized sensors and actuators, while vehicle intelligence and business logic live entirely in central software.*